Northern University of Business and Technology Khulna

Department of Computer Science and Engineering

[10 Week Project Work Plan]

Project Name: Multiplayer LAN Quiz Game

Submitted by:
Md. Shoibe Hossain Rifat
ID: 11240321705
shoiberifat797@gmail.com

Course Title: Software Development I
Course Code: CSE 2216
Section: 4A

Submitted to:
Vashkar Kar[VK]
Lecturer,
Department of Computer Science and Engineering,
Northern University of Business and Technology Khulna

Date:

Signature

PROJECT OVERVIEW

This document outlines a structured 10-week development plan for building a Multiplayer LAN Quiz Game in Java, submitted as the SD-I semester project. The plan covers all learning objectives and implementation tasks week by week, calibrated for a student with intermediate Java knowledge working 5–8 hours per week.

The project is divided into four phases:

- Phase 1 (Weeks 1–3) — Foundation: OOP, file I/O, and basic socket communication
- Phase 2 (Weeks 4–6) — Core features: multithreading, game logic, and Swing UI
- Phase 3 (Weeks 7–9) — Polish: scoreboard, lobby, timer, LAN testing
- Phase 4 (Week 10) — Documentation and final submission

TECHNOLOGY STACK

Component	Technology	Purpose
Language	Java (SE 11+)	Core development
Networking	java.net.ServerSocket / Socket	LAN communication
Concurrency	java.lang.Thread / Runnable	Multi-client handling
UI Framework	Java Swing (javax.swing)	Client-side GUI
Data Storage	Plain text files (.txt)	Questions & scores
IDE	ApacheNetBeans or Eclipse	Development environment

WEEKLY WORK PLAN

PHASE 1 — Foundation **Weeks 1–3**

Week 1	**OOP Review + Project Setup**	~6 hrs

To Learn this week

- Review Java OOP: classes, objects, inheritance, interfaces
- Understand what a Client-Server architecture means , and drawing it on paper
- Read: what is a Socket? What is a Port? What is localhost?

To Implement this week

- Set up IntelliJ IDEA (or Eclipse) with a new Java project
- Create the project folder structure (see File Structure section)
- Write Player.java — name, score, id fields with getters/setters
- Write Question.java — question text, 4 options, correct answer index
- Test: create a Player and Question in Main.java and print them

End of week: *Project compiles. Two model classes exist. You can create a Player and a Question and print them to console.*

Week 2	**File Handling + Quiz Data**	~6 hrs

To Learn this week

- Java File I/O: FileReader, BufferedReader, reading line by line
- How to store quiz questions in a plain text or CSV file
- ArrayList — how to store a list of Question objects in memory

To Implement this week

- Create questions.txt with 15–20 questions in a consistent format
- Write QuestionLoader.java — reads the file, parses each line, returns ArrayList<Question>
- Test: load the file and print all questions to console
- Handle file-not-found error with try-catch

End of week: *All questions load from a file into memory. Error handling works if the file is missing.*

Week 3	**Java Sockets — Single Connection**	~7 hrs

To Learn this week

- ServerSocket and Socket classes in Java (java.net package)
- How to send/receive text over a socket using PrintWriter and BufferedReader
- What a protocol is, define a simple message format, e.g. ANSWER:2, SCORE:15

To Implement this week

- Write Server.java : starts ServerSocket on port 5000, accepts ONE client, sends 'Hello Client'
- Write Client.java : connects to localhost:5000, receives and prints the message
- Extend: server sends a question string, client sends back an answer, server prints it

End of week: *Two programs running on the same machine, communicating over a socket. One question sent and answered end-to-end.*

PHASE 2 — Core Features *Weeks 4–6*

Week 4	**Multithreading — Multiple Clients**	~8 hrs

To Learn this week

- What a Thread is in Java — Thread class and Runnable interface
- Why you need one thread per client (blocking I/O problem)
- Basic thread safety: what a synchronized method does (concept only)

To Implement this week

- Write ClientHandler.java — implements Runnable, manages one client's socket I/O
- Update Server.java — loop that accepts multiple clients and spawns a new thread for each
- Test with 2–3 simultaneous client terminal windows on the same machine

End of week: Server handles 3 clients simultaneously. Each client gets its own thread and receives messages independently.

Week 5	**Quiz Game Logic**	~7 hrs

To Learn this week

- How to design a shared game state: questions, current index, all player scores
- Synchronization: why two threads updating a score simultaneously is a bug, and how synchronized fixes it

To Implement this week

- Write GameManager.java — holds question list, current index, and a Map<String, Integer> of player scores
- Add synchronized submitAnswer(playerName, answer) method — checks correctness and updates score
- Server broadcasts current question to all clients, collects answers, then broadcasts updated scores

End of week: *A full question round works: question sent to all, answers received, scores updated correctly, scores broadcast back to everyone.*

Week 6	**Java Swing UI — Client Side**	~8 hrs

To Learn this week

- Java Swing basics: JFrame, JPanel, JLabel, JButton, JTextField
- Layout managers: BorderLayout and GridLayout are sufficient for this project
- SwingUtilities.invokeLater() , why UI updates must happen on the Event Dispatch Thread

To Implement this week

- Write ClientUI.java , JFrame with a question label, 4 answer buttons, and a scoreboard panel
- Connect UI to socket: when a question arrives, update labels/buttons; when a button is clicked, send the answer
- Add a login screen: name entry field + Connect button that initiates the socket connection

End of week: *Clients connect with a name, see questions as a real GUI (not console), click to answer, and see live scores update.*

PHASE 3 — Polish & Integration Weeks 7–9

Week 7	**Score Tracking + Leaderboard**	~6 hrs

To Implement this week

- Add Scoreboard.java panel : shows all players and scores, sorted highest first
- Server sends a formatted score string after each question, e.g. SCORES:Alice=20,Bob=10
- Parse score string on the client and refresh the scoreboard panel
- Add visual feedback on answer buttons : green/red flash for 1 second after submission
- Save final scores to highscores.txt after the game ends

End of week: *Scoreboard updates live after every question. Visual feedback on answers. Scores saved to file at game end.*

Week 8	**Lobby + Full Game Flow**	~7 hrs

To Implement this week

- Add waiting lobby screen : clients see 'Waiting for host to start...' after connecting
- Server has a simple console command START that begins the game for all connected players
- Add per-question countdown timer (15 seconds) : server tracks time, broadcasts TIME:12, client displays it
- Handle player disconnect gracefully : remove from game without crashing the server
- Add end screen: 'Game Over! Winner: Alice' broadcast to all clients

End of week: *Full game flow works end-to-end: lobby → start → question rounds with timer → end screen. Disconnects are handled cleanly.*

Week 9	**LAN Testing + Bug Fixes**	~7 hrs

To Implement this week

- Find your machine's LAN IP (ipconfig on Windows, ifconfig on Linux/Mac)
- Replace 'localhost' with the actual LAN IP address in Client.java
- Test on at least 2 different physical devices on the same Wi-Fi network
- Fix all bugs found during LAN test : timing issues, UI freezes, message parsing errors
- Add simple input validation: empty name rejected, duplicate name rejected by server
- Add more questions to questions.txt : aim for 30 or more total

End of week: *Game runs cleanly on a real LAN with 2+ devices. No crashes on normal usage paths.*

PHASE 4 — Final Submission Week 10

Week 10	**Documentation + Submission**	~5 hrs

To Implement this week

- Add Javadoc comments to all major classes (/** ... */ above each class and method)
- Write README.txt : how to run the server, how clients connect, what the controls are
- Clean up code: remove debug print statements, organize imports, consistent naming
- Do a final full playthrough with 2 players from start to end screen
- Zip the project folder and submit

End of week: Clean, documented, working project. Ready to demo and submit.

PROJECT FILE STRUCTURE

All source files are organized under a single root folder. The structure below shows every file this project will contain, and what each one does.

File / Folder	Purpose
LanQuizGame/	Root project folder
src/	All Java source files
model/	Data model classes
Player.java	Stores player name, score, and ID
Question.java	Stores question text, 4 options, correct answer index
server/	Server-side classes (run on host machine)
Server.java	Starts ServerSocket, accepts clients in a loop
ClientHandler.java	One thread per connected client — handles I/O
GameManager.java	Shared game state: questions, scores, round logic
client/	Client-side classes (run on each player's machine)
Client.java	Connects to server, sends and receives messages
ClientUI.java	The Swing window the player interacts with
util/	Utility / helper classes
QuestionLoader.java	Reads questions.txt, returns ArrayList<Question>
ScoreFileWriter.java	Writes final scores to highscores.txt
data/	Data files (not source code)
questions.txt	All quiz questions in plain text format
highscores.txt	Written automatically at game end
README.txt	How to run the server and connect clients

questions.txt format

Each question occupies one line. Fields are separated by a pipe character. The last field is the index (0–3) of the correct answer.

```
What is the default port for HTTP?|80|443|8080|8888|0
Which keyword creates a new thread in Java?|new|spawn|thread|start|0
```

Parse each line by splitting on | and mapping the fields to a Question object in QuestionLoader.java.

ARCHITECTURE OVERVIEW

The application follows a Client-Server architecture over a local area network. The server runs on one machine; each player runs the client on their own machine (or the same machine for testing). All communication happens via TCP sockets on port 5000.

Message protocol

All messages are plain strings terminated by a newline. The server and client parse them by splitting on a colon.

Message	Direction	Meaning
NAME:Alice	Client → Server	Player registers their name
QUESTION:What is...?|A|B|C|D	Server → Client	Server sends next question
ANSWER:2	Client → Server	Player submits answer (index 0–3)
SCORES:Alice=20,Bob=10	Server → Client	Updated scoreboard after a round
TIME:12	Server → Client	Seconds remaining on current question
END:Alice	Server → Client	Game over, Alice is the winner

NOTES & STUDY RESOURCES

The following free resources are recommended alongside this plan:

- Oracle Java Docs (docs.oracle.com/javase) : authoritative reference for all java.net, java.io, and javax.swing classes
- 'Java Socket Programming' on YouTube : any beginner tutorial covers Weeks 3–4 in under 2 hours
- 'Java Swing Tutorial' on YouTube : covers Week 6 content well; focus on JFrame layout basics
- GeeksforGeeks Java Multithreading section : clear explanations for the synchronized keyword (Week 5)

A note on pacing: Weeks 3–5 (sockets and multithreading) are the hardest conceptual stretch in this project. Allocate extra time there if needed, and do not hesitate to reread examples. Everything else builds on top of those concepts — once sockets and threads click, the rest comes quickly.